# RAPPORT

## 1. Choix du Langage

Langage retenu : Python

Python offre une excellente compréhension et un développement rapide tout en permettant un contrôle fin des structures de données et des algorithmes.
Ses structures natives (comme les dictionnaires basés sur des tables de hachage) garantissent des recherches en temps constant O(1) pour les identifiants.
Son algorithme de tri interne (**Timsort**) offre une complexité temporelle optimale de O(N log N) dans le pire des cas, idéale pour gérer jusqu'à 100,000 colis.

## 2. Choix Algorithmiques

| **Élément** | **Choix 1 : Recherche par ID** | **Choix 2 : Tri et Affichage** | **Choix 3 : Optimisation du Camion** |
|---|---|---|---|
| **Problème** | Retrouver un colis instantanément parmi 100,000 éléments. | Afficher rapidement les colis classés selon différents critères. | Sélectionner les meilleurs colis sous contrainte de poids max. |
| **Solution** | Utilisation d'un Dictionnaire (Table de hachage). | Utilisation de l'algorithme Timsort natif de Python. | Approche gloutonne basée sur un score composite (Valeur x  Priorité / Poids). |
| **Complexité Temporelle** | O(1) en moyenne | O(N log N) | O(N log N) |
| **Complexité Spatial** | O(N) | O(N) | O(N) |
| **Alternative** | Recherche linéaire dans une liste (O(N)). | Tri par insertion ou sélection ($O(N^2)$). | Programmation dynamique (Sac à dos exact, O(N . W)). |
| **Justification** | Rapidité absolue indispensable pour une interface interactive fluide. | Équilibre parfait entre simplicité d'implémentation et rapidité pour 100,000 éléments. | Compromis optimal entre vitesse d'exécution et pertinence des résultats pour de grands volumes. |

## 3. Préparation du Camion & Prise en Compte de la Priorité

### Problématique

Le problème du chargement du camion s'apparente au problème du sac à dos (0/1 Knapsack Problem), complexifié par l'ajout d'une priorité et d'un passage à l'échelle (100,000 colis). La programmation dynamique classique en temps pseudo-polynomial O(N x W) devient inefficace si la capacité W est très grande ou si N est très élevé.

### Règle de Sélection (Score combiné)

Pour intégrer le poids, la valeur et la priorité sans formule arbitraire figée, il faut définir un indice d'attractivité (score) par colis :

Score = (Valeur x Priorité) / Poids

- On cherche à maximiser la Valeur transportée (numérateur).
- On valorise les colis urgents via la Priorité (multiplicateur au numérateur).
- On pénalise les colis trop lourds qui monopolisent la capacité du camion (divisé par le Poids).

### Stratégie Algorithmique (Approche Gloutonne / Heuristique)

Pour traiter un grand nombre de colis (100,000) de manière fluide :

1. Calculer le score de chaque colis.
2. Trier les colis par ordre décroissant de leur score.
3. Parcourir la liste triée et ajouter les colis au camion tant que la capacité maximale n'est pas dépassée.

**Complexité Temporelle :** O(N log N) due au tri initial des colis.
**Complexité Spatiale :** O(N) pour stocker les scores et la liste triée.
**Alternative :** Programmation dynamique (solution exacte mais trop coûteuse en mémoire et en temps pour 100,000 éléments avec des poids continus ou grands).